

npm notice run retroper@1.0.10 npx npm notice run marked Installation_Guide.md

Retroper — Installation Guide

Retroper is an IDE extension that captures AI usage — prompts, responses, and basic tool activity — from in-editor AI chat panels (Cursor's Composer, VS Code Chat / GitHub Copilot Chat) and from coding agents (Claude Code, Codex CLI), and sends it to a backend for observability / DLP purposes. This guide walks through getting it running, either for local development/testing or as a packaged extension installed into VS Code or Cursor.

This build is not yet published to any marketplace, so it has to be built and installed manually.

1. Prerequisites

Requirement	Notes
Node.js 18 or newer	Needed to install dependencies and compile the extension. Check with <code>node --version</code> .
npm	Ships with Node.js.
VS Code or Cursor (or another VS Code-based IDE)	The extension runs inside any of these, since they share the same extension API.
The Retroper project source	Either the shared folder/zip, or a git clone if you have repo access.
A backend ingest URL	Provided by whoever is running the Retroper backend. Without it the extension will queue records locally but cannot upload them.
A Retroper account (email + password)	Used to log in from inside the extension - see step 5. No token/JWT needs to be shared out of band.

You do **not** need Claude Code or Codex CLI installed for Retroper itself to work - it only captures their activity if and when you happen to use them.

2. Get the source

Copy or clone the `rp-ide-extension` project folder onto your machine, then open a terminal in that folder.

3. Install dependencies

```
npm install
```

4. Configure the backend connection

Copy the example environment file and fill in the real value you were given:

```
cp .env.example .env
```

Open `.env` in a text editor and set:

```
RETROPER_UPLOAD_URL=https://your-backend.example.com/api/v1/ingest/activity
```

`.env` is never committed to source control and is excluded from any packaged build — it stays local to your machine. There is no token/JWT in this file - see the next step.

5. Run it for testing (recommended first step)

1. Open the `rp-ide-extension` folder in VS Code or Cursor.
2. Press **F5** (or Run → Start Debugging). This compiles the extension and opens a second window titled *[Extension Development Host]* with Retroper running inside it.
3. You'll see a notification: **"Retroper: log in to start uploading captured AI usage."** Click **Log In**, then enter your Retroper email and password when prompted. (You can also trigger this any time via the Command Palette → **Retroper: Login**, or by clicking the status bar item in the bottom right that shows your login state.)
4. Once logged in, open the Command Palette (`Ctrl+Shift+P` / `Cmd+Shift+P`) and run:
 - **Retroper: Send Test Record** — sends one fake record through the full pipeline (queue → upload). A confirmation message tells you whether the upload succeeded.
 - **Retroper: Show Capture Status** — shows login state, how many records are currently queued, and where the local queue file is.
 - **Retroper: Flush Queued Records Now** — uploads anything currently queued.
 - **Retroper: Open Audit Log** — opens `audit_log.txt`, a running, timestamped record of every poll, capture, login, and upload attempt. This is the first place to look if anything seems off.
 - **Retroper: Logout** — clears your stored login.

If **Send Test Record** reports success, your setup is correct and the pipeline works end to end.

Your login token is stored in VS Code's encrypted `SecretStorage` (the OS credential store - Windows Credential Manager, macOS Keychain, etc.), not in a plaintext file. It doesn't expire quickly, but if it ever does, uploads will fail once with a clear message and the status bar will flip back to "Not logged in" - just log in again.

6. Install it as a regular extension (optional)

Once you've verified it works via F5, you can package it into a `.vsix` file and install that into your normal VS Code / Cursor — no debug window needed.

```
npm run release
```

This bumps the patch version, compiles, and packages in one step, producing a file like `retroper-1.0.10.vsix` in the project folder — refuses to build if `.env` is missing or `RETROPER_UPLOAD_URL` is blank, so a broken build can't accidentally get shared. (To set an exact version instead of auto-bumping: `npm run release -- 2.0.0` . Plain `npx vsce package` still works too, but skips the version bump and the `.env` sanity check.)

To install the resulting file:

- **VS Code / Cursor UI:** Extensions panel → the `...` menu → *Install from VSIX...* → select the file.
- **Command line:** `code --install-extension retroper-1.0.10.vsix` (or `cursor --install-extension retroper-1.0.10.vsix`).

`.env` is bundled directly into the `.vsix` (just the non-secret `RETROPER_UPLOAD_URL` / `RETROPER_LOGIN_URL` values - no credential lives in it), so there's nothing to configure after installing. Just log in from inside the extension as in step 5.

Upgrading to a newer version

Each version installs into its own separate folder (e.g. `retroper.retroper-1.0.7` and `retroper.retroper-1.0.8` both exist on disk side by side), and installing a new `.vsix` does **not** stop a window that already has the old version running - it only takes effect the next time that window activates.

After installing an upgrade, close or reload every open VS Code / Cursor window (or fully quit and reopen the app). Otherwise an old window keeps running stale code indefinitely in the background, using the same shared local queue and login as the new one - this can look exactly like data being captured or uploaded twice, when what's actually happening is two different versions both doing the same work independently. If you ever see that symptom, check whether an old install folder is still present under your IDE's extensions directory and whether any window still needs a reload.

7. What it currently captures

Source	Surface	Notes
Cursor	Composer / Agent chat	Reads Cursor's own local chat database directly - only works inside Cursor.
VS Code Chat	Built-in Chat panel, including GitHub Copilot Chat	Reads VS Code's own local chat session files - only works inside VS Code (or a fork that ships the same chat feature).

Source	Surface	Notes
Claude Code	CLI and the Claude Code editor tab	Reads Claude Code's own transcript files, which are the same regardless of which editor (or plain terminal) launched it. Works from a single Retroper install, in any IDE.
Codex CLI	CLI and the Codex editor tab	Reads Codex's own session files, same idea - IDE-agnostic, works from a single Retroper install.

Antigravity is not supported yet (not enough real data available to build against). Terminal-based fallback capture for tools without a structured local log isn't implemented.

A provider only captures from the moment it first activates onward. The first time it ever runs, it silently records whatever conversations already exist as "already seen" without uploading them - only new activity from that point forward gets captured. This applies per-provider and per-machine (driven by a local state file), not globally.

8. Troubleshooting

"Retroper: not logged in. Run 'Retroper: Login' to start uploading." Run **Retroper: Login** from the Command Palette, or click the status bar item in the bottom right.

"Retroper: login failed - Email or password is incorrect." Double-check your credentials. This is the backend rejecting them, not an extension bug.

"Retroper: upload failed - Retroper upload not configured" `.env` is missing `RETROPER_UPLOAD_URL`. Re-check step 4.

A record was "permanently rejected by the backend" and dropped The backend's own DLP/content scan blocked that specific record (for example, it detected what looks like a raw secret or token in the captured text) and will never accept it as-is, no matter how many times it's retried. Retroper isolates exactly which record triggered this, drops only that one, and lets everything else in the same batch upload normally - check `audit_log.txt` for the specific record and the backend's reason.

Data looks like it was captured or uploaded twice First check whether an old version of the extension is still running in a window you haven't reloaded since upgrading - see "Upgrading to a newer version" above; this is the most common cause. If every open window is confirmed to be on the same current version and you still see this, check `audit_log.txt` for the exact timestamps and record IDs involved before assuming it's a bug - what looks like a duplicate is sometimes two genuinely different turns with similar content.

If you find the **exact same JSON** reaching the backend more than once (same record `id`, same content), the most likely cause is an upload retry after an ambiguous network failure: if a request actually reaches the backend but the response is lost before the extension sees it, the extension can't tell the difference from a request that never arrived, and retries it - resending the same record `id`. Search `audit_log.txt` for `retrying batch` to confirm whether this happened around the time in question. This can only be fully prevented on the backend, by deduplicating ingest on the record's `id` (or `content_hash_sha256`) field, which every record already carries for exactly this purpose.

Nothing happens when I chat Retroper polls every ~20 seconds and only captures a turn once the AI's response has stopped changing across two consecutive polls (or the turn was aborted/canceled with no response) — it won't appear instantly, and a long streaming or multi-tool-call answer can take longer. Check **Retroper: Show Capture Status** to confirm records are being queued at all.

Where do I see extension logs? Run **Retroper: Open Audit Log**, or open the *Output* panel (View → Output) and select **Retroper** from the dropdown - same content, mirrored to both places. Every poll, capture, login, upload, and error is logged there with a timestamp.

Installed via .vsix and RETROPER_UPLOAD_URL isn't configured This shouldn't happen if the .vsix was built with `npm run release` (it bundles .env in and refuses to build if RETROPER_UPLOAD_URL is blank). If it does, the extension's installed folder is typically:

- Windows: `%USERPROFILE%\vscode\extensions\retroper.retroper-<version>`
- macOS/Linux: `~/.vscode/extensions/retroper.retroper-<version>`

(for Cursor, replace .vscode with .cursor) — check whether .env is present there and has RETROPER_UPLOAD_URL set; create/fix it manually if not, then reload the window.

Still stuck? Check `audit_log.txt` first; most failures (missing config, login errors, upload errors, permanently-rejected records, provider not found) are logged there with a clear message.